گزارش پروژه شماره ۳

(پیاده سازی الگوریتم SPIMI)

درس مبانی بازیابی اطلاعات و جستجوی وب

استاد راهنما: خانم دکتر زهره کریمی

دانشجویان: علی سپهرنیا – امیرعلی مظلومی – مهدی افکار

گروه مهندسی کامپیوتر

دانشکده فنی و مهندسی دانشگاه دامغان

پاییز ۱۴۰۰

فهرست مطالب

مقدمه..... ۳

شرح کلیات پیاده سازی ۳

ساختار پروژه ۵

نگاهی اجمالی بر کدها ۶

ماژول فایل (filesModule.py) ۶

ماژول زبان (langModule.py) ۸

فایل spimi.py ۹

فایل search.py ۱۵

نتایجی از اجرای آزمایشی ۱۷

مقدمه

SPIMI مخفف کلمات Single-pass in-memory indexing است. از این الگوریتم که نسخه بهینه تری از الگوریتم BSBI می باشد برای ساخت شاخص وارون جهت استفاده در موتورهای جستجو استفاده می شود. مزیت اصلی این الگوریتم پیچیدگی زمانی کمتر از مرتبه O(T) می باشد. شبه کد بخش اصلی الگوریتم که به ازای هر بلوک اجرا میشود در شکل زیر قابل مشاهده است:

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

شرح کلیات پیاده سازی

برای پیاده سازی این پروژه از زبان پایتون و کتابخانه ntlk برای توکنایز کردن و ریشه گیری از کلمات استفاده شده است. همان طور که در صورت پروژه آمده است این مجموعه کد قادر است مجموعه ای از اسناد متنی را خوانده، متن هر کدام را توکنایز کرده، کلمات توقف را از آنها حذف کرده و یک لغت نامه از روی آنها بسازد. این لغت نامه قابل تنظیم است و می توان آن را در دو حالت مختلف بدون ریشه گیری و با ریشه گیری از کلمات در نظر گرفت (جهت انعطاف پذیری بیشتر، قابلیت تنظیم برنامه برای حذف یا عدم حذف اعداد از لغت نامه نیز افزوده شده است).

شاخص وارون ساخته شده در این برنامه مطابق مفروضات الگوریتم SPIMI ساخته می شود. با استفاده از یک متغیر تعداد اسناد هر بلوک مشخص می شود و با اجرای الگوریتم، الگوریتم SPIMI-INVERT به ازای هر بلوک به صورت جداگانه اجرا می شود و پس از پایان پردازش هر بلوک، شاخص وارون آن بلوک را در فایلی ذخیره می کند. پس از

اتمام پردازش تمامی فایلها، فرآیند ادغام شاخص های میانی آغاز می شود. در فرآیند ادغام یک اشاره گر خطی در ابتدای هر شاخص میانی قرار می گیرد و با مقایسه کلمه اول همه ی شاخص ها، کوچترین مورد الفبایی را برای ثبت در شاخص نهایی در نظر میگیرد و آن را به همراه لیست پست مربوطه در فایل نهایی مینویسد (البته اگر کلمه تکراری باشد فقط لیست پست مربوطه در شاخص نهایی نوشته می شود). این فرآیند آن قدر تکرار می شود تا به تمامی شاخص های میانی به انتها برسد.

در بخش جستجو، فایل شاخص ساخته شده خوانده شده و با یک تجزیه ساده خطی به یک ساختار داده از نوع dict (بر مبنای کلید-مقدار) نگاشت می شود. به دلیل استفاده از درهم سازی در این ساختمان داده، جستجو در آن بسیار سریع انجام می شود. بعد از دریافت query از کاربر، متن query توکنایز شده، کلمات توقف از آن حذف شده و با توجه به متغیرهای تنظیم شده، عملیات حذف کاراکترهای غیرالفبایی و ریشه گیری انجام می شود. در این بخش علاوه بر امکان جستجوی query یک کلمه ای که در شرح پروژه خواسته شده است، قابلیت جستجوی دو کلمه ای (اسناد شامل هر دو کلمه) نیز فراهم شده است و برنامه مطابق ورودی کاربر تصمیم می گیرد که جستجوی ساده یک کلمه ای یا جستجوی دو کلمه ای انجام شود. در جستجوی تک کلمه ای، لیست پستِ کلمه مربوطه از لغت نامه خوانده شده و به کاربر نمایش داده می شود. در جستجوی دو کلمه ای لیست پست هر کدام از کلمه ها از لغت نامه بازیابی می شود و اشتراک دو لیست پست به عنوان نتیجه برگردانده می شود.

علاوه بر این جهت آگاهی از عملکرد برنامه برخی پارامترها بعد از ساخت شاخص و در هنگام جستجو به کاربر نمایش داده می شود.

این موارد در ساخت شاخص شامل:

- تعداد اسناد پردازش شده
- تعداد بلوک های پردازشی
- تعداد کل کلمات ثبت شده در شاخص نهایی
- مدت زمان ساخت شاخص

و در بخش جستجو شامل:

- مدت زمان بازسازی شاخص
- کلمه (کلمات) حاصل شده از query کاربر
- مدت زمان پیش پردازش query
- مدت زمان جستجو

می باشد.

ساختار پروژه

فایل فشرده پروژه شامل فایل ها و فولدرهای زیر است:

- *data*
  - *
  - *_index*
    - *0*
    - *1*
    - …
    - *30*
    - *docs*
    - *index*
- *toy-data*
  - **.txt*
  - *_index*
    - *0*
    - *1*
    - *docs*
    - *index*
- *filesModule.py*
- *langModule.py*
- *report.pdf*

- *search.py*
- *spimi.py*

فولدرهای data و toy-data شامل فایل های متنی آزمایشی جهت تست پروژه است که روی هر دو فولدر آنها عملیات ساخت شاخص انجام گرفته است و فایل های مربوط به آن در فولدر index1_ و index2_ گردآوری شده است. لازم به ذکر است شاخص موجود در فولدر index1_ در حالت ۱ صورت پروژه (بدون ریشه گیری) و شاخص index2_ در حالت ۲ صورت پروژه (با ریشه گیری از کلمات) ساخته شده است. علاوه بر این در هر دو حالت شاخص، توکن های غیرالفبایی حذف شده است. هر فولدر شاخص علاوه بر فایل شاخص های میانی که با شماره 0 تا n ذخیره شده اند، شامل فایل index و docs نیز می باشد. فایل index همان شاخص نهایی است که به ترتیب حروف الفبا و به ترتیب عددی در هر سطر ابتدا یک کلمه (term) آمده و با یک حرف فاصله (space)، لیستی از شماره اسناد (postings list) نمایش داده شده که در آن هر عدد با یک حرف فاصله از عدد بعدی جدا شده است. فایل docs شامل جدولی از نگاشت اسم سند به شماره سند است. در واقع در هر سطر آن ابتدا نام سند آمده و با یک حرف فاصله بعد آن شماره سند ذکر شده است.

فایل های اسکریپت برنامه شامل دو فایل filesModule.py (ماژول فایل) و langModule.py (ماژول زبان) به عنوان ماژول های برنامه و فایل های search.py و spimi.py به عنوان فایل های اصلی برنامه می باشد. ماژول فایل شامل توابع کمکی برای کار با فایل و متن است و ماژول زبان دارای توابع کمکی برای انجام عملیات های زبانی روی متن. spimi.py علاوه بر الگوریتم و توابع مربوط به آن شامل پارامترهای تنظیم الگوریتم و فراخوانی تابع ساخت شاخص است.

نگاهی اجمالی بر کدها

ماژول فایل (filesModule.py)

کدهای موجود در این فایل به شرح زیر است:

```
import os

def list_files(directory, doc_dic):
```

```
    list = []
    # Walk the tree.
    for root, dirs, files in os.walk(directory):
        for filename in files:
            list.append(os.path.join(root, filename))
            doc_dic.write(f'{filename} {len(list) - 1}\n')
        break
    doc_dic.close()
    return list
```

این تابع با گرفتن آدرس یک فولدر (directory) و یک شئ io stream لیستی از تمام فایل های موجود در آن فولدر را بر می گرداند. این تابع ضمن لیست کردن فایل ها، یک به یک نام آنها به اضافه شماره سند آنها (0 تا n) را در فایلی که در شیء doc_dic باز است می نویسد.

```
def get_file_content(filepath):
    with open(filepath, 'r') as file:
        file_content = file.read()
    return file_content
```

این تابع محتویات فایل به آدرس filepath را بر میگرداند

```
def parse_index_line(text):
    brk = text.index(' ')
    # Break text to two parts (term and postings)
    parts = [text[:brk], text[brk+1:]]
    return parts[0], parts[1].removesuffix('\n')
```

این تابع محتویات یک خط از شاخص وارون (text) را می گیرد و آن را به زوج لغت و لیست پست میشکند.

```
def parse_index(path):
    dic = {}
    with open(path, 'r') as file:
        for line in file.readlines():
            term, post = parse_index_line(line)
            dic[term] = post
    return dic
```

این تابع محتویات فایل شاخص وارون واقع در آدرس path را خوانده و آن را به یک ساختمان داده از نوع dict تجزیه میکند.

```
def open_write(root, ifolder, fname):
    make_dir(root, ifolder)
    path = os.path.join(root, ifolder, str(fname))
    return open(path, 'w')
```

این تابع آدرس فولدر ریشه (root)، نام فولدر داخلی (ifolder) و نام فایل مورد نظر را میگیرد و در آن آدرس یک فایل جدید برای نوشتن ایجاد میکند و نهایتا شیء io stream مربوط به آن را برمیگرداند.

```
def open_reads(root, ifolder, max):
    list = []
    for n in range(0, max):
        path = os.path.join(root, ifolder, str(n))
        file = open(path, 'r')
        list.append(file)
    return list
```

این تابع تمامی فایل های موجود در فولدر root و فولدر داخلی ifolder که نام آنها از 0 شروع شده و تا max ادامه دارد را به منظور خواندن باز میکند و لیستی از اشیاء io stream مربوط به آن فایلها را برمیگرداند.

```
def clear_dir(root, ifolder):
    path = os.path.join(root, ifolder)
    if os.path.exists(path):
        for file in os.listdir(path):
            os.remove(os.path.join(path, file))
        os.rmdir(path)
```

این تابع در صورتی که فولدر ifolder در فولدر root وجود داشته باشد، تمامی فایل های موجود در ifolder و در نهایت خود آن پوشه را حذف می کند.

```
def make_dir(root, folder):
    path = os.path.join(root, folder)
    if not os.path.exists(path):
        os.mkdir(path)
```

این تابع در صورتی که فولدر folder در فولدر root وجود نداشته باشد، آن را ایجاد میکند.

ماژول زبان (langModule.py)

کدهای موجود در این ماژول به شرح زیر است:

```
import nltk
from nltk.stem import PorterStemmer

stop_words = list(nltk.corpus.stopwords.words('english'))
stemmer = PorterStemmer()

def get_tokens(text, alpha=True):
    tokens = nltk.word_tokenize(text)
    if alpha:
```

```
        return [token.lower() for token in tokens if token.isalpha()]
    return [token.lower() for token in tokens if token.isalnum()]
```

این تابع با دریافت متن ورودی (text) آن را توکنایز کرده و لیست کلمات آن را برمیگرداند. پارامتر alpha تعیین میکند که توکن های شامل حروف غیر الفبایی از لیست حذف شوند (alpha=True) یا نه (alpha=False).

```
def remove_stop_words(tokens):
    return [item for item in tokens if item not in stop_words]
```

این تابع لیستی از کلمات را دریافت میکند و لیستی از کلمات بدون کلمات توقف را برمیگرداند.

```
def get_stems(tokens):
    return [stemmer.stem(token) for token in tokens]
```

این تابع لیستی از کلمات را دریافت میکند و لیستی از ریشه لغوی آن کلمات را برمیگرداند.

```
def proccess_text(text, stemming=False, alpha=True):
    tokens = get_tokens(text, alpha)
    tokens = remove_stop_words(tokens)
    if stemming:
        tokens = get_stems(tokens)
    return tokens
```

این تابع یک متن (text) را دریافت میکند و لیستی از توکن های آن را برمیگرداند. اگر stemming=True باشد ریشه کلمات نیز گرفته میشود. اگر apha=True باشد، توکن های شامل حروف غیرالفبایی از خروجی حذف میشوند.

```
def get_term_docIDs(text, docID, stemming=False, alpha=True):
    tokens = proccess_text(text, stemming, alpha)
    return [(token, docID) for token in tokens]
```

این تابع همان تابع قبل را روی متن ورودی (text) اجرا میکند و در انتها لیستی از زوج مرتب های شامل لغت و شماره سند (docID) را برمیگرداند.

فایل spimi.py

این فایل شامل بخش زیادی از کدهای اصلی برنامه است و برای جلوگیری از شلوغ شدن گزارش از ذکر کل کد به صورت یکجا خودداری می شود. عملکرد بخش های مهم این کد به شرح زیر است:

```
# Config Vars
_root, _ifolder = r'data', '_index'
_block_size, _post_init_size = 5, 3
```

```
_stemming, _alpha = True, True
```

متغیر _root آدرس فولدر دیتا که شامل اسناد برای ساخت شاخص است را نگهداری میکند و متغیر _ifolder نام فولدری که فایل های شاخص در آن ذخیره میشوند. متغیر _block_size و متغیر _post_init_size به ترتیب تعداد اسناد در هر بلوک و سایز اولیه لیست پست ها را ذخیره میکند. متغیرهای _stemming و _alpha نیز تعیین میکنند که ریشه گیری و حذف توکن های شامل حروف غیرالفبایی نیز صورت بگیرد یا خیر.

```
files.clear_dir(_root, _ifolder) # delete old index files
```

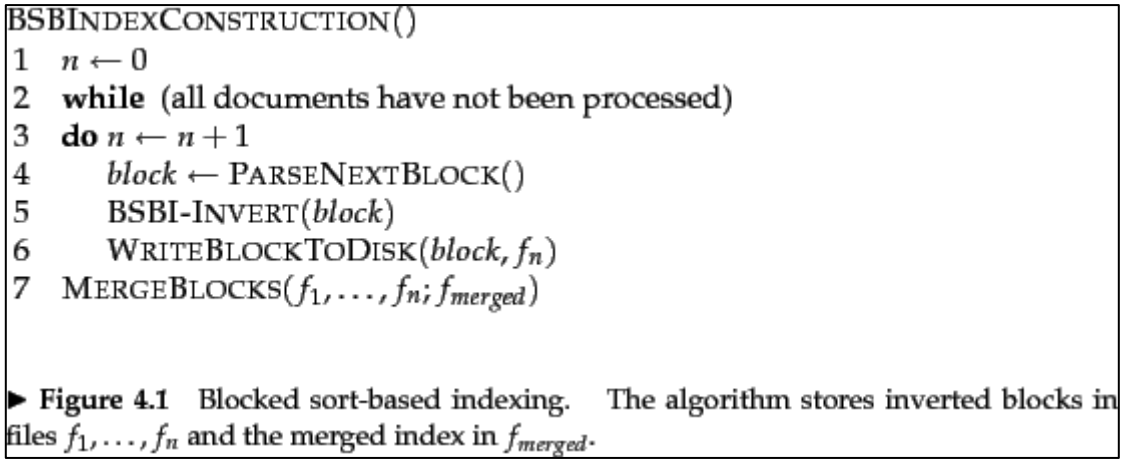
این خط تابعی را از ماژول فایل فراخوانی می کند تا تمامی فایل های ساخته شده در شاخص قبلی را در صورت وجود پاکسازی کند.

```
# Global Vars
_files_list = files.list_files(_root, files.open_write(_root, _ifolder, 'docs'))
_index, n = 0, 0
_posts_size = {}
```

متغیر اول با فراخوانی تابعی از ماژول فایل، لیستی از فایل های موجود در فولدر ریشه را دریافت می کند و نگاشت نام سند-شماره سند ها را نیز در فایل ‘docs’ کنار شاخص ها ذخیره میکند. _index یک اشاره گر به شماره سند است و n یک شمارشگر است که تعدا بلوک های پردازش شده را حفظ می کند. _posts_size شامل یک ساختمان داده خالی از نوع dict برای نگهداری از اندازه لیست پست هاست.

```
def SPIMI_INDEX_CONSTRUCTION(stemming, alpha):
    global n
    n = 0
    while(_index < len(_files_list)):
        token_stream = PARSE_NEXT_BLOCK(stemming, alpha)
        SPIMI_INVERT(token_stream)
        n = n + 1
    MERGE_BLOCKS(f_list(n), f('index'))
```

این تابع مشابه شبه کد ساخت شاخص به روش BSBI است که در کتاب آورده شده است و دقیقا همان کار را انجام میدهد.


BSBINDEXCONSTRUCTION()
1 $n \leftarrow 0$
2 **while** (all documents have not been processed)
3 **do** $n \leftarrow n + 1$
4 $block \leftarrow$ PARSENEXTBLOCK()
5 BSBI-INVERT($block$)
6 WRITEBLOCKTODISK($block, f_n$)
7 MERGEBLOCKS($f_1, \ldots, f_n; f_{merged}$)

▶ **Figure 4.1** Blocked sort-based indexing. The algorithm stores inverted blocks in files $f_1, \ldots, f_n$ and the merged index in f_{merged}.

```
def PARSE_NEXT_BLOCK(stemming, alpha):
    global _index
    block = []
    stop = min(_index + _block_size, len(_files_list))
    for i in range(_index, stop):
        file_content = files.get_file_content(_files_list[i])
        term_docIDs = lang.get_term_docIDs(file_content, i, stemming, alpha)
        block.extend(term_docIDs)
    _index = _index + _block_size
    return block
```

این تابع که در تابع ساخت شاخص صدا زده می شود به این صورت عمل میکند:

به اشاره گر شماره سند به اندازه تعداد اسناد در بلوک اضافه کن و اگر هنوز اشاره گر خارج از محدوده نرفته بود به

ازای شمارش از اشاره گر شماره سند تا کران بالای شماره سند این کار را تکرار کن:

محتویات فایل این شمارش را بخوان، به زوج های لغت-شماره سند تبدیل کن و این زوج ها در کنار زوج

های قبلی بریز.

در پایان لیست زوج های لغت-شماره سند را برگردان.

```
def SPIMI_INVERT(token_stream):
    output_file = NEW_FILE() #= output_file ← NEW_FILE()
    dictionary = {} #= dictionary ← NEW_HASH()
    while(True): #= while(free memory available)
        for token in token_stream: #= do token ← next(token_stream)
            term, docID = token[0], token[1]
            if term not in dictionary: #= if term(token) ∉ dictionary then
                #= postings_list ← ADD_TO_DICTIONARY(dictionary, term(token))
                postings_list = dictionary[term] = []
            else:
```

```
                #= postings_list ← GET_POSTINGS_LIST(dictionary, term(token))
                postings_list = dictionary[term]
            if full(postings_list, term): #= if full(postings_list) then
            #= postings_list ← DOUBLE_POSTINGS_LIST(dictionary, term(token))
                _posts_size[term] *= 2
            #= ADD_TO_POSTINGS_LIST(postings_list, docID)
            postings_list.append(docID)
        break # To stop while from infinite loop!
    #= sorted_terms ← SORT_TERMS(dictionary)
    sorted_terms = sorted(dictionary.items(), key=lambda x: x[0])
    WRITE_BLOCK_TO_DISK(sorted_terms, dictionary, output_file)
    return output_file
```

این تابع کاملا مشابه شبه کد الگوریتم شاخص وارون SPIMI در کتاب است:

```
SPIMI-INVERT(token_stream)
 1  output_file = NEWFILE()
 2  dictionary = NEWHASH()
 3  while (free memory available)
 4  do token ← next(token_stream)
 5     if term(token) ∉ dictionary
 6        then postings_list = ADDTODICTIONARY(dictionary, term(token))
 7        else postings_list = GETPOSTINGSLIST(dictionary, term(token))
 8     if full(postings_list)
 9        then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11  sorted_terms ← SORTTERMS(dictionary)
12  WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13  return output_file
```

*) برای دو برابر کردن سایز پست لیست ها از یک ساختمان داده dict استفاده شده که در آن لغت مدنظر و اندازه لیست پست مربوط به آن نگهداری میشود و در صورت پر شدن آن اندازه دو برابر می شود. این کار از آنجایی انجام می شود که در پایتون اندازه ساختمان داده لیست که برای لیست پست ها استفاده میشود پویا است و نیازی به دوبرابر کردن ندارد و این شیوه تنها برای رعایت مراحل تعریف شده در کتاب انجام می شود.

```
def MERGE_BLOCKS(input_files, output_file):
    curr_lines = [file.readline().removesuffix('\n') for file in input_files]
    prev_term, curr_term = None, None
    first = True

    while len(input_files) > 0:
        m = min(curr_lines, key=lambda l:(l.split(' ')[0], int(l.split(' ')[1])))
        p = curr_lines.index(m) # Index of doc with smallest term and smallest postings
        # Parse term and postings_list from current line
        curr_term, last_posts = files.parse_index_line(min(curr_lines))
```

```
        if curr_term != prev_term:
            if first:
                first = False
            else:
                # Go to new line if its not the first line
                output_file.write('\n')
            output_file.write(f'{curr_term} {last_posts}')
            prev_term = curr_term
        else:
            output_file.write(f' {last_posts}')

        nwline = input_files[p].readline()
        curr_lines[p] = nwline
        if nwline == '' or nwline == '\n' :
            input_files[p].close()
            input_files.pop(p)
            curr_lines.pop(p)

    output_file.close()
```

این تابع که برای ادغام شاخص های میانی در شاخص نهایی به کار می رود، مطابق با روند توضیح داده شده در کتاب و در کلاس عمل می کند. ابتدا از همه ی شاخص های میانی یکی یک خط می خواند و در لیستی نگهداری میکند. تا زمانی که فایلی برای خواندن وجود داشته باشد روند زیر را تکرار میکند:

کوچکترین خط الفبایی را پیدا میکند (شامل زوج کوچکترین کلمه و کوچکترین عدد در اول لیست پست) و آن را حفظ میکند. لغت فعلی و لیست پست مربوطه را از روی همان خط به دست می آورد. در صورتی که لغت فعلی همان لغت قبلی نباشد، در شاخص نهایی لغت و لیست پست را با هم درج میکند. در غیر این صورت یعنی اگر لغت برای بار چندم تکرار شد تنها لیست پست آن را در شاخص نهایی درج می کند. در همین فایل یک خط به جلو حرکت میکند، محتویات آن خط را خوانده و در لیست آخرین خطوط خوانده شده، اندیس مربوط به آن شاخص میانی را با خط جدید بروزرسانی میکند. اگر خط جدید خالی بود، آن شاخص را میبندد، شاخص را از لیست فایل ها و خط خوانده شده را از لیست آخرین خط ها حذف میکند.

```
def full(list, term):
    global _posts_size
    if term in _posts_size:
        return len(list) >= _posts_size[term]
    else:
        _posts_size[term] = _post_init_size
        return False
```

این تابع با دریافت لیست پست و لغت مربوطه، بررسی می کند که آیا اندازه آن لیست از اندازه تعریف شده برای آن تجاوز کرده است یا نه. همچنین اگر لغت مربوطه برای اولین بار دیده شده است، آن را به آرایه سایزها اضافه میکند و اندازه اولیه آن را برابر متغیرهای اولیه تنظیم میکند.

```
def WRITE_BLOCK_TO_DISK(sorted_terms, dictionary, output_file):
    for term, postings in sorted_terms:
        output = f'{term} ' + ' '.join([str(i) for i in postings]) + '\n'
        output_file.write(output)
    output_file.close()
```

این تابع لیستی مرتب از توکن ها به همراه لیست پست آنها را دریافت می کند و مطابق فرمت تعریف شده در فایل خروجی مینویسد.

*) فرمت خروجی در هر خط: "لغت شماره_سند شماره_سند شماره_سند ... شماره_سند"

```
def NEW_FILE():
    files.make_dir(_root, _ifolder)
    return files.open_write(_root, _ifolder, str(n))
```

این تابع یک فولدر جدید در فولدر ریشه به نام _ifolder و یک فایل جدید به نام n (شماره آخرین بلوک) برای نوشتن می سازد و شئ io stream آن را برمیگرداند.

```
#fn
def f(n):
    return files.open_write(_root, _ifolder, n)
```

این تابع شئ io stream برای نوشتن در فایل n ام در فولدر شاخص را بر میگرداند.

```
#f1,...,fn
def f_list(max):
    return files.open_reads(_root, _ifolder, max)
```

این تابع شماره آخرین بلوک (max) را گرفته و لیست کل io stream های شاخص های میانی را برای خواندن برمیگرداند.

در نهایت الگوریتم ساخت شاخص به روش SPIMI فراخوانی می شود و پارامترهای کارکردی الگوریتم برای کاربر در کنسول نمایش داده می شود که کدهای آن به صورت زیر است:

```
start = time.perf_counter()
SPIMI_INDEX_CONSTRUCTION(_stemming, _alpha)
end = time.perf_counter()

print('//Number of files proccessed: ', len(_files_list))
print('//Number of blocks proccessed: ', n)
lines_num = 0
with open(os.path.join(_root, _ifolder, 'index'), 'r') as file:
        lines_num = len(file.readlines())
print('//Total words in dictionary: ', lines_num)
print('//Index construction time: ', ((end - start) * 1000), 'ms')
```

فایل search.py

کدهای موجود در این فایل به شرح زیر است:

```
import os
import time
import filesModule as files
import langModule as lang

# Config Vars
_root, _ifolder = r'data', '_index'
_index_path = os.path.join(_root, _ifolder, 'index')
_stemming, _alpha = True, True

def search_two_terms(list):
    term1, term2 = list[0], list[1]
    if term1 in dic and term2 in dic:
        s1 = set(dic[term1].split(' '))
        s2 = set(dic[term2].split(' '))
        result = ' '.join(sorted(s1.intersection(s2)))
        if len(result) > 0:
            return result
    return None
```

این تابع نتیجه جستجوی دو کلمه ای (لیستی از شماره اسناد که شامل هر دو کلمه اند) را برمیگرداند. برای این کار ابتدا بررسی میکند که هر دو کلمه در لغت نامه موجود باشند (اگر نبود Null برمیگرداند). سپس یک مجموعه از هر کدام از لیست پست های مربوط به آن کلمه میسازد و از آن دو مجموعه اشتراک میگیرد. بین هر کدام از عناصر

مجموعه یک کاراکتر فاصله قرار میدهد و رشته حاصل را اگر خالی نبود بر میگرداند. ((اگر خالی بود Null برمیگرداند)

```
start = time.perf_counter()
dic = files.parse_index(_index_path)
end = time.perf_counter()
print('Index reconstruction time: ', ((end - start) * 1000), 'ms', '\n')
```

این قسمت از کد، فایل شاخص نهایی را به تابع تجزیه شاخص از ماژول فایل میدهد تا آن را به یک لغت نامه (dict) تبدیل کند. مدت زمان صرف شده برای این کار به کاربر گزارش می شود.

```
query = input("Enter one/two word(s) to search for:\t")
start = time.perf_counter()
terms = lang.proccess_text(query, _stemming, _alpha)
end = time.perf_counter()
print('//Query parse time: ', ((end - start) * 1000), 'ms', '\n')
```

از کاربر میخواهد query خود شامل یک یا دو کلمه را در محیط کنسول وارد کند. با استفاده از تابع پردازش متن از ماژول زبان و طبق پارامترهای تنظیمی، متن را به لغات تجزیه می کند. مدت زمان صرف شده برای این کار به کاربر گزارش می شود.

```
start = time.perf_counter()
if len(terms) > 0:
    two = False
    result = None
    if len(terms) >= 2:
        two = True
        terms = [terms[0], terms[1]]
        result = search_two_terms(terms)
    else:
        result = dic[terms[0]]
    end = time.perf_counter()
    print('Document ids witch contains ', f'{terms}:\n {result}', '\n')
    print('//Search time: ', ((end - start) * 1000), 'ms', '\n')
else:
    print("Enter meaningful words to search for!")
```

این قسمت شامل فرآیند نهایی جستجو است. اگر کوئری شامل کلماتی نباشید، پیغام خطایی را به کاربر نمایش میدهد. در غیر این صورت اگر لغات query حداقل ۲ تا بود، با تابع جستجوی دو کلمه ای به دنبال اسناد مربوط این لغات میگردد و نتیجه را در متغیری ذخیره میکند و در غیر اینصورت تنها پست لیست مربوط به تک لغت query را

در متغیر نتیجه ذخیره میکند. در پایان علاوه بر نمایش لغات حاصل شده از پردازش query و شماره اسناد مربوط، دیگر پارامترهای کارکردی جستجو نیز به کاربر نمایش داده می شود.

نتایجی از اجرای آزمایشی

جهت تکمیل و به عنوان حسن ختام این گزارش، مختصری از نتایج اجرای این پروژه در این جا ذکر میشود. این نتایج در برابر اجرای برنامه برای دیتای آزمایشی تعریف شده در صورت پروژه (پوشه data) و در سیستمی با مشخصات زیر حاصل شده است:

Model	ASUS X542UF
Processor	Intel(R) Core(TM) i5-8250U CPU
RAM	8GB DDR4 (1200MHz)
Disk	WD Green SATA SSD M.2 2280
OS	Windows 10 Pro 21H1
IDE	Visual Studio Code 1.63.0
Python	Python 3.10.1

خروجی کنسول در ساخت شاخص با حذف توکن های غیرالفبایی و ریشه گیری:

```
//Number of files proccessed:  155
//Number of blocks proccessed:  31
//Total words in dictionary:  3784
//Index construction time:  1672.5742999836802 ms
```

خروجی کنسول در ساخت شاخص با حذف توکن های غیرالفبایی و بدون ریشه گیری:

```
//Number of files proccessed:  155
//Number of blocks proccessed:  31
//Total words in dictionary:  5076
//Index construction time:  851.0584000032395 ms
```

خروجی کنسول در جستجوی لغت ’wins‘ با حذف توکن های غیرالفبایی و ریشه گیری:

```
Index reconstruction time:  4.165700054727495 ms


Enter one/two word(s) to search for:    information
//Query parse time:  5.94569998793304 ms


Document ids witch contains  ['inform']:
 1 2 2 11 28 45 49 51 59 61 61 63 70 71 71 74 76 82 85 93 94 94 97 120 120 124
128 128 129 129 129 129 130 130 131 132 133 133 133 134 134 134 135 135 135 135
135 136 136 136 137 139 140 140 140 140 143 143 143 143 144 145 151 151 152


//Search time:  0.003500026650726795 ms
```

خروجی کنسول در پرس و جوی ’visiting a website‘ با حذف توکن های غیرالفبایی و ریشه گیری:

```
Index reconstruction time:  4.36769996304065 ms


Enter one/two word(s) to search for:    visiting a website
//Query parse time:  6.256949963141233 ms


Document ids witch contains  ['visit', 'websit']:
 128 129 130 131 132 133 134 135 136 138 139 140 143 144 145 146 147 148 149 150
151 152 153 154


//Search time:  0.0397998579174284 ms
```

خروجی کنسول در پرس و جوی ’visiting a website‘ با حذف توکن های غیرالفبایی و بدون ریشه گیری:

```
//Index reconstruction time:  5.148700089193881 ms


Enter one/two word(s) to search for:    visiting a website
//Query parse time:  5.955899961665273 ms


Document ids witch contains  ['visiting', 'website']:
 128 129 130 131 132 133 134 135 136 138 139 140 143 144 145 146 147 148 149 150
151 152 153 154
```

```
//Search time:  0.03680005118995905 ms
```

*) همگی مدت زمان ها در خروجی کنسول، میانگین ۱۰ بار اجرا به صورت دستی است.